FootyEdge AI — Production Betting Intelligence Platform Blueprint

Executive Summary

FootyEdge AI should not be built as a "football prediction app."

It should be built as a:

Betting Intelligence Platform

A system that collects football data, generates probabilistic forecasts, identifies market inefficiencies, applies bankroll optimization, validates performance through backtesting, and distributes actionable betting signals.

The objective is not prediction accuracy alone.

The objective is:

Long-term positive expected value (EV) and sustainable ROI.

System Vision

Core Mission

Transform raw football data into:

- Match probabilities
- Value bets
- Sharp-money insights
- Risk-adjusted staking recommendations
- Premium betting signals

Architecture Overview

```
        ┌──────────────────────────┐
        │    API Gateway     │
        │     FastAPI        │
        └────────────┬─────────────┘
            │
  ┌──────────────────────┼──────────────────────┐
  │              │          │
  ▼              ▼          ▼

Data Service     Prediction Engine   Market Engine
(Data Layer)      (AI Layer)        (Value Layer)

  │              │          │
  └──────────────────┬───┼──────────┬─────────┘
             │        │
             ▼        ▼

       Risk Engine   Analytics Engine

             │
             ▼

       Signal Service

             │
             ▼

    Telegram / Web / API Clients

             │
             ▼

       Training Pipeline
```

Module 1: Data Service

Responsibility

Collect and normalize football data.

Sources:

Primary

- API-Football (RapidAPI)

Secondary

- Football-Data.org

Enhancement

- 365Scores
- SofaScore

Required Endpoints

get_matches()
get_odds()
get_team_stats()
get_h2h()
get_standings()
get_player_stats()

Data Normalization

Convert all sources into a single format.

Example:

```
{
  "fixture_id": 12345,
  "home_team": "Arsenal",
  "away_team": "Chelsea",
  "date": "2026-06-14",
  "league": "Premier League"
```

}

Never expose raw provider responses to downstream services.

Caching Layer

Avoid API exhaustion.

Level 1

Memory Cache

cache = {}

TTL:

3600 seconds

Level 2

Redis (Future)

Module 2: Feature Engineering Engine

This layer converts football data into ML features.

Team Form Features

Last:

- 5 matches
- 10 matches
- 20 matches

Metrics:

win_rate
draw_rate
loss_rate
goals_scored_avg
goals_conceded_avg

Home/Away Features

home_win_rate
away_win_rate
home_xg
away_xg

Head-to-Head Features

h2h_home_wins
h2h_draws
h2h_away_wins

League Strength

league_rank
league_points
goal_difference

Player Features

Injury score:

injury_weight

Suspensions:

availability_score

Top scorer contribution:

goal_contribution_ratio

Module 3: Team Strength Agent

Current architecture already includes:

TeamStrengthAgent

Upgrade to:

attack_strength
defense_strength
form_strength
home_strength
away_strength
squad_strength

Example Output

```
{
  "attack": 1.45,
  "defense": 0.82,
  "form": 0.78
}
```

Module 4: Tactical Agent

Purpose:

Detect stylistic matchups.

Examples:

High Press Team

vs

Possession Team

May alter xG expectations.

Outputs:

```
{
  "tempo": "high",
  "matchup_advantage": 0.08
}
```

Module 5: Player Impact Agent

Purpose:

Adjust probabilities.

Example:

If Haaland unavailable:

home_xg -= 0.35

If goalkeeper absent:

away_xg += 0.20

Module 6: Hybrid Prediction Engine

Current system uses Poisson.

Keep it.

But combine with ML.

Layer A

Poisson

Produces:

xG
score matrix
goal probabilities

Layer B

Machine Learning

Model:

LightGBM

or

XGBoost

Features:

100–300 engineered features.

Outputs:

home_win
draw
away_win

Hybrid Blend

final_prob =
0.6 * ml_prob +
0.4 * poisson_prob

Module 7: Probability Engine

Current implementation:

```
for h in range(10):
    for a in range(10):
```

Keep.

Increase:

range(12)

for better accuracy.

Generate:

home_win
draw
away_win

Goal markets:

over_1_5
over_2_5
over_3_5

BTTS:

yes
no

Double chance:

home_draw
away_draw
home_away

Module 8: Value Bet Engine

Core formula:

EV = (Probability × Odds) − 1

Thresholds:

EV > 0.03

Value

EV > 0.07

Strong

EV > 0.12

Premium

Example

Probability = 0.60
Odds = 2.10

EV = 0.26

26% edge.

Module 9: Sharp Money Detector

Purpose:

Track bookmaker movements.

Input

opening_odds
current_odds

Detection

drop > 8%

Flag:

Sharp Money

Output

```
{
  "signal": "Sharp Inflow",
  "strength": 0.14
}
```

Module 10: Kelly Risk Engine

Never release without this.

Formula

Where:

b = odds - 1
p = probability
q = 1-p

Implementation

stake =
bankroll * kelly_fraction

Risk Controls

Maximum:

25% Kelly

Recommended:

50% Kelly Fraction

Module 11: Backtesting Engine

Purpose:

Validate profitability.

Metrics

ROI

roi

Yield

yield

Win Rate

win_rate

Drawdown

max_drawdown

Profit Factor

profit_factor

Never deploy live before:

5000+
historical matches

Module 12: ML Training Pipeline

Directory:

ml_pipeline/

dataset_builder.py
train.py
evaluate.py
export.py

Workflow

Collect Data
↓
Build Features
↓
Train Model
↓
Evaluate
↓
Export Model

Training Models

Phase 1

LightGBM

Phase 2

XGBoost

Phase 3

Neural Network

Module 13: Database Design

Use

Tables

teams

id
name
country
league

matches

id
home_team
away_team
date

---

predictions

id
fixture_id
home_prob
draw_prob
away_prob
confidence

---

value_bets

id
market
selection
odds
ev
status

---

user_bets

id
stake
odds
result

---

model_runs

id
version
accuracy
roi

Module 14: Signal Engine

Outputs:

Free

Low EV

Premium

EV > 0.07

Elite

EV > 0.12 + Sharp Confirmation

Message Example

⚽ Arsenal vs Chelsea

🎯 Over 2.5 Goals
📈 EV: 12%

💰 Recommended Stake:
2.5% Bankroll

🔥 Premium Signal

Module 15: Telegram Automation

Use official Telegram Bot API.

Store:

TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=

Flows:

Prediction
↓
Value Bet
↓
Kelly
↓
Telegram

Module 16: SaaS Layer

Free:

- 3 picks/day

Premium:

- Full signals
- Kelly stakes
- Sharp alerts

Elite:

- API access
- Portfolio tracking

Module 17: Monitoring

Track:

prediction_accuracy
roi
api_errors
telegram_failures

Tools

- Render logs
- Supabase logs
- Sentry (future)

Module 18: Deployment Strategy

Phase 1

Single Render instance

FastAPI
Supabase
Telegram

Phase 2

Split services

prediction-service
market-service
signal-service

Phase 3

Containerization

Use:

Phase 4

Kubernetes only if:

100k+ users

Critical Recommendations

Immediate

1. Remove simulated betting outputs in production.
2. Decouple predictor from database access.
3. Create dedicated feature-engineering layer.
4. Add Kelly staking before any signal release.
5. Build backtesting before selling subscriptions.

Short Term

1. Train LightGBM model.
2. Build historical dataset.
3. Add odds movement tracking.
4. Add result settlement automation.

Long Term

1. Multi-bookmaker aggregation.
2. Closing Line Value (CLV) tracking.
3. Arbitrage detection.
4. Reinforcement learning stake optimization.
5. Automated portfolio management.

Success Metric

Do not judge success by prediction accuracy.

Judge success by:

Positive EV
+
Positive ROI
+
Controlled Drawdown
+
Consistent Bankroll Growth

That is the benchmark a professional betting intelligence platform should optimize for.